29. Performance Basics: CPU, memory, iowait, OOM

29. Performance Basics: CPU, memory, iowait, OOM

Phase: Phase 10 — Observability and troubleshooting
Estimated time: 2 days
Slug: `29-perf-basics`

Learning objectives

- Read load average correctly
- Spot memory pressure and OOM
- Distinguish CPU-bound vs IO-bound
- Use vmstat/iostat at a basic level

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

"The server is slow" is not a diagnosis. Performance work on Linux means finding which of four resources is saturated: **CPU**, **memory**, **disk I/O**, or **network**. Each one has a different signature and a different fix. Guessing wastes hours; a five-minute structured look usually finds the bottleneck. This module also covers the **OOM killer**, which silently ends processes when memory runs out and is a frequent cause of "random" service crashes.

Core concepts

Resource	Quick tool	Signature of saturation
CPU	`uptime`, `top`, `mpstat`	load above core count, high `%us`/`%sy`, run queue long
Memory	`free -h`, `vmstat`	low `available`, swap in/out (`si`/`so`), OOM kills
Disk I/O	`iostat -x`, `iotop`	high `%util`, high `await`, high `%wa` in top
Network	`ss -s`, `sar -n DEV`	drops, retransmits, link at line rate

Load average (1, 5, 15 minute) counts tasks that are running **or waiting for CPU or uninterruptible I/O**. Compare it with the number of CPUs:

```
load < cpus    : headroom
load ~ cpus    : fully busy, little slack
load > cpus    : tasks are queueing (CPU or D-state I/O wait)
```

`iowait` (`%wa`) is CPU time spent idle while disk requests are outstanding. A high value means "waiting on storage", not "CPU is busy".

```
request --> CPU --> page cache --hit--> done
                        |
                       miss
                        v
                  block layer --> disk --> done   (this wait shows as iowait)
```

Linux uses spare RAM as page cache, so low “free” is normal. Look at **available** instead.

Worked examples

CPU count and load, then decide if busy (the exercise logic):

```
nproc
uptime
# load_busy CPUS LOAD -> yes if LOAD > CPUS
awk -v c=2 -v l=4 'BEGIN{print (l>c)?"yes":"no"}'
```

Who burns CPU right now:

```
top -b -n1 | head -n 15
ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head
```

Memory view:

```
free -h
vmstat 1 5
grep -E 'MemAvailable|SwapFree' /proc/meminfo
```

In `vmstat`, watch columns: `r` (runnable), `b` (blocked), `si/so` (swap), `bi/bo` (blocks in/out), `wa` (iowait).

Disk view:

```
iostat -x 1 3
iotop -o        # needs root
```

Find an OOM kill after the fact:

```
journalctl -k | grep -i -E 'out of memory|oom'
dmesg -T | grep -i 'killed process'
```

Typical kernel line:

```
Out of memory: Killed process 4121 (java) total-vm:8123456kB, anon-rss:3912000kB
```

Per-process memory and OOM score:

```
ps -eo pid,comm,rss --sort=-rss | head
cat /proc/$$/oom_score
```

Create controlled load for practice:

```
timeout 10 sh -c 'while :; do :; done' &
uptime
```

Mini-lab

1. Record `nproc` and `uptime`. Decide: is the load above the CPU count?
2. Start two busy loops as above (one per core is enough to see effect). Watch `top` and note %us and load growth over a minute.
3. Run `vmstat 1 5` idle, then during a heavy `dd if=/dev/zero of=/tmp/x bs=1M count=500 oflag=dsync`. Compare `bo` and `wa`.
4. Run `free -h`. Explain the difference between `free` and `available`.
5. Search the kernel log for OOM lines. If none, explain what you would look for.
6. Clean up: `kill %1 %2`, `rm -f /tmp/x`.

Decision guide / failure modes

Observation	Likely cause	Next step
load high, `%us` high	CPU-bound process	`top`, profile app, scale out
load high, CPU idle, `%wa` high	storage wait	`iostat -x`, check disk, NFS, noisy neighbor
`si/so` nonzero constantly	memory pressure, swapping	find big RSS, add RAM or limit
process vanished, no app error	OOM kill	check `dmesg`/journal, set limits
load high, many `D` state procs	stuck I/O (NFS, dying disk)	`ps -eo stat,comm | grep '^D'`
`%sy` high	syscalls, network, context switching	`strace -c`, `perf top`

Failure modes to watch for:

- Reading load without knowing the CPU count. Load 4 is fine on 16 cores and a disaster on 1.
- Treating `iowait` as CPU use. It is idle time that is blocked on I/O.
- Panicking over low `free` memory when `available` is large.
- Fixing the symptom: raising swap hides a leak. Find the growing RSS.
- Container limits: a cgroup can OOM-kill a process while the host has plenty of free RAM. Check `docker inspect` memory limits and `memory.events`.
- One-shot measurements: sample over time (`vmstat 1`, `iostat 1`). A single snapshot misses bursts.

Common mistakes

- Misreading load average

- Disabling OOM killer
- Optimizing CPU while disk is the bottleneck

Exercises

Work in `exercises/29-perf-basics/`. Read that folder's `README.md` fully before coding.

```
cd exercises/29-perf-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `load_busy CPUS LOAD` in `exercise.sh`. It must print `yes` when the load average is strictly greater than the CPU count and `no` otherwise (equal means not busy). Load values can be decimals such as 2.5, so use `awk` rather than shell integer math. Run `./test.sh`, then try your function with the real values from `nproc` and `uptime` on your machine. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/29-perf-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Load 4 on 2 CPUs means?
2. How spot OOM?
3. iowait suggests what?

Further reading

- https://www.brendangregg.com/linuxperf.html